Experimental Protocol

For each trial, collect evidence from:

- **GOOSE layer:** decoded value (PCAP/Wireshark or your subscriber logs)
- **IED/app layer:** subscriber output showing buffer value and sequence fields
- **SCADA/HMI layer:** topology state change / breaker state indicator
- **Control-center interface:** IEC-104 telemetry

Repeatability: 10–30 trials with real rigor

1) Decide your "trial recipe" (keep it identical across runs)

Pick fixed parameters:

- Trial duration: e.g., **120 s**
- Warm-up: e.g., **20 s** (ignore for stats)
- Injection start: e.g., **t = 30 s**
- Sampling/logging rate: e.g., **10 Hz** (or whatever your logs allow)

2) Define conditions (you need at least two)

- **C0 Baseline:** no injection
- **C1 Step FDI:** abrupt change to injected value (~10)

Run **N = 10–30 trials per condition** (e.g., 20 each is a nice midpoint).

3) Define success criteria (binary outcomes per trial)

This is what turns a demo into an experiment.

A trial is "successful" if ALL hold:

1. Baseline window: buffer is within nominal band (you will estimate this band empirically)
2. After injection: buffer enters injected band near ~10 and stays there for ≥ X seconds
3. SCADA/HMI reflects corresponding state (breaker-open indication)

Write down concrete thresholds, e.g.:

- Nominal band: $\mu_0 \pm 3\sigma_0$from baseline calibration
- Injected band: $9.5, 10.5$(or data-driven)

4) What to report (minimum tables/plots)

Table 1 – Repeatability summary

- Condition (C0, C1)
- **trials**
- Mean buffer (baseline window)
- Std dev buffer
- Success rate (%)
- Time-to-state-change at HMI (if measurable)

Figure 1 – Time series overlay

- Plot buffer vs time for several trials (thin lines), and the mean (thick line).
- Show baseline vs step-FDI in separate panels or separate figures.

Statistics

- Report effect size: difference in means ($\Delta \approx 5$)
- Report a CI: 95% CI for baseline mean and injected mean
- If you want one statistical test: Welch's t-test baseline vs injected window (not required, but nice)

5) Reduce confounds (small steps, big credibility)

- Reset simulator state between trials (same initial breaker state).
- Randomize trial order (baseline vs attack) to avoid drift in the environment.
- Log versions + configuration + host load (basic reproducibility).

Stealth variant: gradual drift vs sudden jump

1) Define at least 2 drift profiles

Add two new conditions:

- **C2 Linear ramp:** buffer increases from 5 → 10 over **T_ramp** seconds
    - Example: T_ramp ∈ {30, 60, 120}
- **C3 Slow random-walk drift:** buffer increments by small steps with noise but a positive trend
    - Constrained to plausible rate-of-change and bounds

(Even C2 alone is enough, but C3 is great if you have time.)

2) Define the stealth objective

State what the attacker is optimizing:

- Keep rate-of-change below typical anomaly thresholds
- Still cross the logic threshold that triggers "oil temp too high / breaker open"

3) What to measure

For each drift trial:

- Time when buffer crosses the "trip interpretation threshold"
- Whether SCADA/HMI changes state
- Whether your detector triggers (next section)
- Detection latency (likely longer)

How to present results

Figure 2 – Step vs drift time series

- Show one representative step trial and one drift trial (same axis scale).
- In the caption, call out that drift stays under derivative-based thresholds longer.

Table 2 – Stealthiness metrics

- Condition (Step, Ramp-30s, Ramp-60s, Ramp-120s)
- Trip time (median)
- Detector trigger rate (%)
- Detection latency (median, IQR)

Defense evaluation: simple invariant checks + latency + false positives

You want a detector that is:

- **Streaming** (works in real time)
- **Calibrated on baseline**
- Evaluated on both step and drift

1) Detector #1: rate-of-change (RoC) threshold

Compute discrete derivative:

$$d_t = \frac{x_t - x_{t-1}}{\Delta t}$$

Alarm rule:

- Trigger if | d_t |> $\tau$for **k consecutive samples** (k reduces noise alarms)

Calibration (important):

- Use baseline trials only.
- Set τ as a high percentile of |d_t| during baseline (e.g., 99.9th percentile).
- Report τ in the paper (reviewers like to see it's data-driven).

Expected outcome:

- Step attacks trigger fast (low latency)
- Drift attacks may evade (higher latency or missed detections)

2) Detector #2: correlation / cross-signal invariant

You need a second signal that *should* move with "oil temperature too high / breaker open" interpretation.

Pick one:

Option A (best): physical plausibility / consistency

- If breaker is "open", then **current/power flow should drop** (or change materially).
- Invariant: if "over-temp / open" is reported but power-flow indicator doesn't change ⇒ suspicious.

Option B: redundant measurement consistency

- Compare temperature-derived flag vs a second temperature tag or related sensor proxy.

Option C: temporal consistency

- Over-temperature should not appear without a warm-up trend; sudden flags without precursor slope are suspicious.

A simple correlation-style rule (windowed):

- Compute correlation between buffer and a related signal over a sliding window W.
- If correlation deviates strongly from baseline distribution ⇒ alarm.

You can also do a very simple **rule-based** invariant:

- If buffer indicates "open/over-temp" AND (related signal stays in nominal band) for > T seconds ⇒ alarm.

3) Evaluation protocol

You need **both** false positives and detection latency.

Definitions (state them in the paper):

- **t_attack_start:** time injection begins (known in testbed)
- **t_alarm:** first time detector triggers
- **Detection latency:** $t_{alarm} - t_{attack\_start}$
- **False positive (FP):** alarm triggered in a baseline trial
- **FP rate:** alarms per minute/hour OR % of baseline trials with any alarm

Run detectors on:

- Baseline trials (C0) → estimate FP
- Step FDI trials (C1) → estimate detection rate + latency
- Drift trials (C2/C3) → estimate detection rate + latency degradation

4) What to report

Table 3 – Defense performance
For each condition and detector:

- Detection rate (% trials detected)

- Median latency (and IQR)
- FP rate (alarms/hour) or (% baseline trials flagged)

Figure 3 – Latency distribution

- A box plot of latency for step vs drift (separate detectors)

Optional (extra credit):

- ROC-style sweep: vary τ and plot TPR vs FPR (even 5–10 points is fine)

Practical experiment checklist

Per trial (repeatable run sheet)

1. Reset simulator to known state
2. Start logging:
    - Subscriber logs (buffer + timestamps + sequence fields)
    - PCAP (for evidence; not necessarily for detector)
    - HMI snapshot or logged state transitions
3. Wait warm-up period
4. Apply condition (none / step / ramp / random walk)
5. Continue until the end time
6. Store outputs in a structured folder:
    - trial_XX/condition_C1/subscriber.log
    - trial_XX/condition_C1/capture.pcap
    - trial_XX/condition_C1/hmi.png
    - trial_XX/condition_C1/meta.json (start time, config hash)

After all trials

- Parse into a single dataset (CSV): time, buffer, condition, trial_id, breaker_state, etc.
- Compute summary stats + detection outputs.
- Generate the 3 tables + 3 figures above.

In Simple Terms

Choose **N = 10 to 30** independent trials.

Each trial consists of:

1. Baseline capture (no attack)
2. Jump attack capture (attack script executed)
3. Drift attack

Naming convention:

- trial_001 … trial_030

3) Per-trial capture procedure (repeat N times)

3.1 Trial initialization

For trial trial_k:

1. Start SGSim and wait until the scenario is stable (topology up, IEDs publishing).
2. Wait **warm-up** time (recommend **30 s**) to avoid startup transients.
3. Ensure no other background experiments are running.

Record in lab_notes.md:

- trial ID
- start time
- any anomalies observed (packet drops, simulator restarts, etc.)

3.2 Baseline capture (pcapng)

1. Start capture on the **same interface each time**:
    a. Prefer a point where GOOSE flows are visible (subscriber-side interface or switch port mirror).
2. Capture duration: **T = 30–60 s** (pick one and keep constant).
3. Stop capture and save as:

data/trials/trial_k/baseline.pcapng

Sanity check after capture (quick):

- File exists and size is non-trivial (> a few MB depending on traffic).

3.3 Jump attack capture (pcapng)

1. Start capture again (same interface, same duration T).

2. Start the attack script that triggers the SGSim jump behavior.
3. Let it run for the full capture window.
4. Stop capture and save:

data/trials/trial_k/attack.pcapng

Trial acceptance criteria:

- baseline.pcapng and attack.pcapng both present
- each includes GOOSE frames for the selected goID (checked later automatically)

If a trial fails (e.g., no GOOSE, corrupted pcap, simulator crash):

- Keep folder but mark trial as invalid in lab_notes.md and rerun as trial_k_retake or skip.

4) Build a drift variant

Drift can be generated from each baseline automatically by the pipeline if you struggle to implement, but the protocol should define it clearly (you can let me know):

Default drift parameters (stealth):

- shift = +5.0 (so ~5 → ~10)
- start = 2 s
- duration = 10 s
- model = sigmoid

This creates:

data/trials_csv_drift/trial_k/baseline.csv

data/trials_csv_drift/trial_k/attack.csv # drifted version